CPL: Language Design and Formal-ish Specification

Techical report

Fot Nikolaj

03.25.2026

Abstract

CPL is a low-level experimental programming language intended for compiler construction, systems programming experiments, and educational use. Its design emphasizes explicit control flow, compact implementation, direct access to low-level mechanisms, and a deliberately small core language. The language is statically typed in a practical but permissive sense: it provides explicit types and compile-time checking, while still allowing a number of implementation-level implicit conversions. CPL does not attempt to provide memory safety. Instead, it exposes unsafe facilities such as raw pointers, explicit dereferencing, inline assembly, target-facing storage control, and manual interaction with the calling convention.

This document presents a formal-ish language report for the current state of CPL. It specifies the core syntax, type system, storage model, control-flow constructs, function model, preprocessing facilities, static diagnostics, and the current status of implementation-defined and target-dependent behavior.

1 Introduction

CPL is designed as a small low-level language for exploring compiler architecture and code generation. It is not intended to be a general-purpose high-level language. In particular, it intentionally omits user-defined aggregate abstractions such as `struct`, `class`, `enum`, and `union`, and instead focuses on primitive values, arrays, strings, pointers, functions, and explicit statements.

The current implementation favors:

- explicitness over abstraction,
- direct control over generated code,
- support for backend-oriented experimentation,
- a compact frontend and semantic model,
- a compiler architecture suitable for further PL and compiler research.

CPL should therefore be read not as a safe systems language, but as a deliberately exposed low-level language in which the programmer remains responsible for many semantic invariants.

2 Conformance and Status

This document is a **formal-ish** specification rather than a fully formal semantics. Some behaviors are fixed by design, while others remain implementation-defined, target-dependent, or intentionally left unspecified.

The document describes the language **as currently implemented and intended**, including a number of design choices that may later be revised.

The following terminology is used throughout:

- **well-defined**: behavior fixed by this report,
- **implementation-defined**: behavior chosen by the compiler implementation and expected to be documented,
- **target-dependent**: behavior determined by the target ABI, machine model, or backend,
- **undefined behavior (UB)**: behavior for which the language imposes no semantic guarantee.

3 Translation Model

A CPL program is processed as a translation unit after preprocessing. A program may consist of multiple source files combined through preprocessing includes and external declarations.

A source file may contain:

- preprocessing directives,
- imports,
- external declarations,
- global declarations,
- function prototypes,
- function definitions,
- a `start` entry function.

A file without an entry point may be treated as a library unit.

4 Lexical Structure

4.1 Tokens

The implementation distinguishes at least the following lexical categories:

- identifiers,
- integer literals,
- floating-point literals,
- string literals,
- character literals,
- comments,
- end-of-line markers for preprocessing.

The following token names are assumed at the lexical level:

- `IDENTIFIER`
- `INTEGER_LITERAL`
- `FLOAT_LITERAL`

- STRING_LITERAL
- CHAR_LITERAL
- COMMENT
- EOL

4.2 Comments

Comments are supported as lexical elements and do not affect program semantics.

4.3 Keywords

The current language reserves at least the following keywords:

extern, glob, ro, function, start, arr, if, else, loop, while, switch, case, default, return, exit, break, lis, syscall, asm, not, ref, dref, as, ptr, str, f64, i64, u64, f32, i32, u32, i16, u16, i8, u8, line, include, define, undef, ifdef, ifndef, endif, section, align.

4.4 Literals

The following literal classes exist:

- integer literals,
- floating-point literals,
- string literals,
- character literals.

Every integer literal is typed as i64. Every floating-point literal is typed as f64.
This rule is semantic rather than merely lexical.

5 Concrete Syntax

The following EBNF describes the current surface syntax of the language. It is based on the current parser design and may omit some annotation-related syntax and lambda-specific sugar.

```
identifier       = "IDENTIFIER" ;
integer_literal  = "INTEGER_LITERAL" ;
float_literal    = "FLOAT_LITERAL" ;
string_literal   = "STRING_LITERAL" ;
char_literal     = "CHAR_LITERAL" ;
comment          = "COMMENT" ;
eol              = "EOL" ;

program        = "{" , { top_item } , "}" ;

top_item       = start_function
               | import_op
               | extern_op
               | pp_directive
               | storage_opt , top_decl ;

import_op      = "from" , string_literal , [ import_list ] ;
import_list    = import_item , { "," , import_item } ;
import_item    = identifier ;
```

```
extern_op      = "extern" , ( extern_function_proto | var_prototype ) ;
extern_function_proto
               = "function" , identifier , "(" , [ param_list ] , ")" , [ "->" , type
    ] , ";" ;
var_prototype  = type , identifier ;

storage_opt    = [ "glob" | "ro" ] ;

top_decl       = declaration | function_def | function_proto ;

declaration    = var_decl | arr_decl ;

function_def   = "function" , identifier , "(" , [ param_list ] , ")" , [ "->" , type
    ] , block ;
function_proto = "function" , identifier , "(" , [ param_list ] , ")" , [ "->" , type
    ] , ";" ;
start_function = "start" , "(" , [ param_list ] , ")" , block ;

param_list     = param , { "," , param } ;
param          = type , identifier , [ "=" , expression ] | "..." ;

block          = "{" , { block_item } , "}" ;

block_item     = statement
               | function_def
               | function_proto ;

statement      =
    "arr" , arr_stmt_tail
  | pp_directive
  | if_statement
  | loop_statement
  | while_statement
  | switch_statement
  | return_statement
  | exit_statement
  | break_statement
  | lis_statement
  | syscall_statement
  | asm_block
  | align_stmt
  | comment
  | block
  | var_decl_starting_not_arr
  | expression_statement ;

arr_stmt_tail  =
    identifier , "[" , integer_literal , "," , type , "]" ,
    [ "=" , ( expression | arr_value ) ] , ";"
  | "[" , integer_literal , "," , type , "]" , identifier ,
    [ "=" , expression ] , ";" ;

pp_directive   = "#" , pp_body , pp_end ;
pp_end         = eol | ";" ;
pp_body        = pp_line
               | pp_include
               | pp_define
```

```
                | pp_undef
                | pp_ifdef
                | pp_ifndef
                | pp_endif ;

pp_line         = "line" , integer_literal , [ string_literal ] ;
pp_include      = "include" , string_literal ;
pp_define       = "define" , identifier , { pp_token } ;
pp_undef        = "undef" , identifier ;
pp_ifdef        = "ifdef" , identifier ;
pp_ifndef       = "ifndef" , identifier ;
pp_endif        = "endif" ;

pp_token        = identifier
                | literal
                | keyword
                | punct_no_semi
                | operator ;

keyword         = "extern" | "exfunc" | "glob" | "ro"
                | "function" | "start"
                | "arr"
                | "if" | "else" | "loop" | "while"
                | "switch" | "case" | "default"
                | "return" | "exit" | "break" | "lis"
                | "syscall" | "asm"
                | "not" | "ref" | "dref" | "as"
                | "ptr" | "str"
                | "f64" | "i64" | "u64" | "f32" | "i32" | "u32" | "i16" | "u16" | "i8"
    | "u8"
                | "line" | "include" | "define" | "undef" | "ifdef" | "ifndef" | "endif
    " ;

punct_no_semi   = "{" | "}" | "(" | ")" | "[" | "]" | "," ;

operator        = "->"
                | "==" | "!=" | "<=" | ">="
                | "<<" | ">>"
                | "||" | "&&"
                | "+=" | "-=" | "*=" | "/=" | "%=" | "|=" | "^=" | "&=" | "||=" | "&&="
                | "="  | "<"  | ">"  | "+"  | "-"  | "*"  | "/"  | "%"  | "|"  | "^"  |
     "&" ;

var_decl        = type , identifier , [ "=" , expression ] , ";" ;

var_decl_starting_not_arr
                = non_arr_type , identifier , [ "=" , expression ] , ";" ;

non_arr_type    = "f64" | "i64" | "u64" | "f32" | "i32" | "u32" | "i16" | "u16" | "i8"
    | "u8"
                | "str"
                | "ptr" , type ;

arr_decl        = "arr" , identifier , "[" , integer_literal , "," , type , "]" ,
                  [ "=" , ( expression | arr_value ) ] , ";" ;

arr_value       = "{" , [ arr_value_list ] , "}" ;
arr_value_list = expression , { "," , expression } ;
```

```
if_statement   = "if" , expression , ";" , statement , [ "else" , statement ] ;
loop_statement = "loop" , statement ;
while_statement
               = "while" , expression , ";" , statement ;

switch_statement
               = "switch" , expression , ";" ,
                 "{" , { case_block } , [ default_block ] , "}" ;
case_block     = "case" , literal , ";" , block ;
default_block  = "default" , [ ";" ] , block ;

return_statement
               = "return" , [ expression ] , ";" ;
exit_statement = "exit" , expression , ";" ;
break_statement
               = "break" , ";" ;
lis_statement  = "lis" , [ string_literal ] , ";" ;

expression_statement
               = expression , ";" ;

syscall_statement
               = "syscall" , "(" , [ expression_list ] , ")" , ";" ;
expression_list
               = expression , { "," , expression } ;

asm_block      = "asm" , "(" , [ asm_args ] , ")" , "{" , { asm_line } , "}" ;
asm_args       = asm_arg , { "," , asm_arg } ;
asm_arg        = identifier | literal ;
asm_line       = string_literal , [ "," ] ;

expression     = assign ;

assign         = logical_or , [ assign_op , assign ] ;
assign_op      = "=" | "+=" | "-=" | "*=" | "/=" | "%=" | "|=" | "^=" | "&=" | "||=" |
     "&&=" ;

logical_or     = logical_and , { "||" , logical_and } ;
logical_and    = bit_or      , { "&&" , bit_or } ;

bit_or         = bit_xor     , { "|"  , bit_xor } ;
bit_xor        = bit_and     , { "^"  , bit_and } ;
bit_and        = equality    , { "&"  , equality } ;

equality       = relational  , { ( "==" | "!=" ) , relational } ;
relational     = shift       , { ( "<" | "<=" | ">" | ">=" ) , shift } ;
shift          = add         , { ( "<<" | ">>" ) , add } ;
add            = mul         , { ( "+" | "-" ) , mul } ;
mul            = unary       , { ( "*" | "/" | "%" ) , unary } ;

unary          = unary_op , unary
               | postfix ;

unary_op       = "not" | "+" | "-" | ref_op | dref_op ;
ref_op         = "ref" ;
dref_op        = "dref" ;
```

```
postfix        = primary , { postfix_op } ;
postfix_op     = "(" , [ arg_list ] , ")"
               | "[" , expression , { "," , expression } , "]"
               | "as" , type ;

primary        = literal
               | identifier
               | "(" , expression , ")" ;

arg_list       = expression , { "," , expression } ;

type           = "f64" | "i64" | "u64" | "f32" | "i32" | "u32" | "i16" | "u16" | "i8"
    | "u8"
               | "str"
               | "arr" , "[" , integer_literal , "," , type , "]"
               | "ptr" , type ;

literal        = integer_literal | float_literal | string_literal | char_literal ;
```

5.1 Notes on Syntax Status

The grammar above reflects the current parser structure, but the following must be noted:

- annotation syntax is not fully represented here,
- lambda syntax is not yet specified in the grammar,
- some historical operators may remain in the parser or grammar,
- `arr a[10, i32]` is the canonical array declaration form.

6 Type System

6.1 Overview

CPL is statically typed, but not strictly in the sense of a sound high-level type system. Types are explicit and checked, yet the implementation permits a number of implicit conversions. The language currently includes:

- primitive numeric types,
- a string wrapper type,
- arrays,
- pointers.

6.2 Primitive Numeric Types

The following primitive numeric types are supported:

- signed integers: `i8`, `i16`, `i32`, `i64`
- unsigned integers: `u8`, `u16`, `u32`, `u64`
- floating-point types: `f32`, `f64`

  These types are first-class scalar values.

6.3 The `i0` Type

The special type `i0` is used as a no-value or void-like return type. A function declared with `-> i0` may use `return;` but may not use `return expr;`.

A function without an explicit return type does **not** mean `i0`. Instead, such a function is treated as returning `i64` by default.

6.4 String Type

`str` is currently a wrapper over a character array, conceptually close to C-style `char[]`. It exists partly as a placeholder for future string-related work, but in the current language it should be understood as a character array abstraction rather than as a separate immutable or managed string object.

Important properties of `str` in the current design:

- it is **not** read-only by definition,
- it is not a high-level string object,
- in function parameters it is passed by reference rather than by value,
- semantically it behaves as a wrapper around array-like storage of characters.

6.5 Arrays

Arrays are declared using the form:

```
arr a[10, i32];
```

An array has:

- a fixed constant length,
- an element type,
- storage allocated according to its declaration context.

There is currently **no implicit array-to-pointer decay**. A function expecting a reference or pointer to an array must be called explicitly with `ref a` where appropriate.

6.6 Pointers

Pointer types are written as:

```
ptr T
```

where `T` is another type.

Nested pointer types are permitted:

```
ptr i32
ptr ptr i32
ptr str
```

Pointers expose raw memory-level programming. CPL does not provide ownership, borrowing, or safety tracking.

7 Storage Classes and Visibility

7.1 Local Storage

Variables declared inside functions without storage modifiers are local objects, normally placed in stack-based storage.

7.2 `glob`

When applied to a variable, including primitive variables, arrays, and strings, `glob` means the object is not stored on the stack but in a section-visible static storage area. Such an object is visible throughout the file.

When applied to a function, `glob` means the function may be externally visible.

7.3 `ro`

`ro` marks an object as read-only storage. Read-only objects may exist both for primitives and aggregate-like values such as arrays and strings.

A variable declared outside a function must appear as `glob` or `ro`.

7.4 Top-level Declarations

At top level, variables may only exist as `glob` or `ro`. Functions may be declared or defined at top level and may also use external visibility controls.

8 Scope and Lifetime

8.1 Lexical Scope

CPL uses lexical block scope. A declaration is visible from its point of declaration within its block and in nested blocks, unless shadowed by another declaration.

8.2 Lifetime of Local Objects

Local objects live until the end of their enclosing scope. Using a reference or pointer to such an object after the end of its lifetime is undefined behavior.

8.3 Reference-taking

The `ref` operator forms a reference-like address to an object. Because CPL does not provide formal lvalue/rvalue categories in the frontend specification, the language relies on operational restrictions: only valid addressable program entities may be referenced.

8.4 No Safe Escape Tracking

The language does not enforce safe escape analysis at the user level. A programmer may construct dangling references or pointers; such behavior is invalid.

9 Expressions

9.1 Expression Classes

CPL expressions include:

- literals,

- identifiers,
- unary operations,
- binary operations,
- casts,
- indexing,
- calls,
- assignments.

Lambda expressions are implemented as syntactic sugar for hidden local functions and are therefore semantically lowered rather than directly represented as a separate runtime value class.

9.2 Literal Typing

The following literal typing rules hold:

- every integer literal has type `i64`,
- every floating-point literal has type `f64`.

9.3 Conditions

Conditions in `if`, `while`, and similar constructs must be integer-like values. Floating-point values are **not** valid condition values.

9.4 Operators and Precedence

The effective precedence structure is:

1. postfix operations: call, indexing, cast
2. unary operations: `not`, unary `+`, unary `-`, `ref`, `dref`
3. multiplicative: `*`, `/`, `%`
4. additive: `+`, `-`
5. shifts: «, »
6. relational: `<`, `<=`, `>`, `>=`
7. equality: `==`, `!=`
8. bitwise `&`
9. bitwise `^`
10. bitwise `|`
11. logical `&&`
12. logical `||`
13. assignment and update assignment

Associativity rules are:

- most binary arithmetic, comparison, bitwise, indexing, and call operations associate left to right,
- unary operators associate right to left,
- assignments and update assignments associate right to left,
- `as` is treated left to right as a postfix cast form.

9.5 Unary Operators

The current unary operators are:

- `not`
- unary `+`
- unary `-`
- `ref`
- `dref`

9.6 Casts

Casts use postfix syntax:

```
x as i32
```

The cast operator is explicit and participates at postfix precedence.

10 Implicit Conversions

10.1 General Status

CPL currently allows implicit conversions according to implementation rules rather than a fully principled source-level conversion lattice. The language is therefore best described as having a **permissive static conversion model**.

10.2 Current Implementation Rule

The current compiler allows an implicit conversion when the source and target are sufficiently similar and when pointer/reference levels satisfy the implementation's admissibility condition. In practical terms, conversions are often allowed when:

- the underlying subject types are considered similar,
- the pointer level matches, or
- both source and target fall into categories where full-machine-width representation makes explicit conversion unnecessary.

This is not yet a polished language-lawyer rule. It is an implementation-level rule that should later be refined into a proper specification table.

10.3 Consequence

As a result:

- not all implicit conversions are semantically clean,
- the language report can document the current behavior,
- but future revisions should tighten this area significantly.

11 Statements and Control Flow

11.1 Blocks

A block introduces a new lexical scope and contains zero or more block items.

11.2 Variable Declarations

Variables may be declared either through ordinary typed declarations or array declarations.

Examples:

```
i32 x = 10;
str s = "hello";
arr a[10, i32];
```

11.3 `if`

The `if` statement has the form:

```
if expr; statement
else statement
```

The semicolon after the condition is part of the syntax.

11.4 `while`

The `while` statement has the form:

```
while expr; statement
```

The condition must be non-floating-point.

11.5 `loop`

`loop` expresses an unconditional loop:

```
loop statement
```

It is typically terminated through `break` or other control transfer.

11.6 `switch`

The `switch` statement selects among literal-valued cases. Cases are block-based.

```
switch expr; {
    case 1; { ... }
    case 2; { ... }
    default; { ... }
}
```

11.7 `return`

`return` may appear in ordinary functions. Its allowed form depends on the return type:

- for `-> i0`, only `return;` is valid,
- for ordinary typed returns, `return expr;` is expected,
- for functions with omitted return type, the operational default is `i64`.

11.8 `exit`

`exit expr;` terminates the `start` function path. A `start` function is expected to use `exit` rather than ordinary typed returns.

11.9 `break`

`break;` exits the nearest enclosing breakable construct.

12 Functions

12.1 Ordinary Functions

Functions are declared as:

```
function name(params) -> type { ... }
```

or as prototypes:

```
function name(params) -> type;
```

If the return type is omitted, the function defaults to returning `i64`.

12.2 Parameters

Function parameters are typed and may optionally include default expressions.

12.3 Variadic Parameters

Variadic functions are supported through `...` in the parameter list.

12.4 Parameter Passing

Primitive values are passed by value. Non-primitive entities, including `str`, are not passed by value in the implementation model. `str` parameters therefore behave as reference-style parameters.

12.5 Local Functions

Functions may appear within blocks. Such functions are local in scope.

12.6 Lambdas

Lambdas are lowered into hidden local functions and passed by reference. They are therefore not modeled as closure objects. They do not capture outer environments.

13 Entry Points

The canonical entry construct is:

```
start(...) { ... }
```

A `start` function serves as the program entry point.

Its completion is expected to occur through `exit expr;`.

Other entry-related mechanisms may exist in the implementation, but `start` is the primary language-level entry construct.

14 Low-level and Target-facing Constructs

14.1 `extern`

`extern` declares externally provided functions or variables.

14.2 `syscall`

`syscall(...)` performs a target-facing system call operation. Its detailed behavior is target-dependent.

14.3 `asm`

Inline assembly is written as:

```
asm(args...) {
    "instruction"
    "instruction"
}
```

Its semantics depend on the target backend and assembly syntax. The compiler does not attempt to make inline assembly safe.

14.4 `lis`

`lis` exists as a statement form in the grammar. Its exact high-level semantic role should be documented separately if it remains part of the stable language surface.

15 Annotations

The current report does not fully formalize annotation syntax, but annotations are part of the language design and may affect declarations, code generation, and control-flow lowering.

15.1 `hot` and `cold`

`hot` and `cold` are intentional source-level annotations that affect lowering and placement strategy. Their semantics are part of the design on purpose and are not treated as accidental backend artifacts.

In the current version, they are considered semantically meaningful layout-influencing constructs, not bugs.

15.2 Other Annotations

The implementation also refers to annotations such as alignment, section placement, and related low-level controls. These should eventually receive a dedicated normative subsection.

16 Static Diagnostics

16.1 General Structure

The compiler performs static diagnostics, and some diagnostics are blocking.

16.2 Blocking Diagnostics

The following categories are blocking according to the current implementation:

- assignment to read-only storage,
- missing required return in ordinary functions,
- missing required `exit` in `start`,
- not enough call arguments,
- illegal array access,
- invalid `exit` placement or use,
- assigning the result of a no-return-value call.

  These are effectively compile-time errors.

16.3 Non-blocking Diagnostics

Other diagnostics may exist as warnings or advisory checks, but only the blocking classes above are guaranteed here as hard failures in the current implementation.

17 Well-defined, Implementation-defined, and Undefined Behavior

17.1 Well-defined Behavior

The following are part of the current intended language semantics:

- integer literals are `i64`,
- floating literals are `f64`,
- floating-point values are not valid conditions,
- `str` is a mutable wrapper over character-array storage,
- arrays do not decay implicitly to pointers,
- omitted function return type defaults to `i64`,
- `-> i0` functions may not return a value expression,
- lambdas lower to hidden local functions passed by reference.

17.2 Implementation-defined Behavior

The following are still implementation-defined or incompletely regularized:

- the full admissibility table for implicit conversions,
- exact annotation syntax in the grammar,
- some interaction details between low-level storage modifiers and code generation,
- some semantics of historical or rarely documented statements such as `lis`.

17.3 Target-dependent Behavior

The following depend on the compilation target or backend:

- pointer size,
- calling convention,
- external symbol linkage details,
- inline assembly semantics,
- system call ABI,
- section layout.

17.4 Undefined Behavior

The following are undefined behavior:

- using references or pointers to dead local storage,
- violating the semantic assumptions of low-level unsafe operations,
- any invalid memory access through raw pointers,
- backend-invalid or ABI-invalid use of target-facing constructs such as `asm` and `syscall`.

18 Design Summary

CPL is a compact, explicit, and intentionally unsafe low-level language. It should be understood as a compiler-oriented experimental language rather than a production systems language. Its semantics are already structured enough to support a report-style presentation, but several parts still reflect implementation pragmatics more than a finished language design.

Its strongest current characteristics are:

- small core syntax,
- explicit low-level operations,
- direct support for compiler experimentation,
- clear separation between primitive values and manually managed memory,
- deliberate rejection of automatic high-level abstractions.

Its weakest current area is the conversion model, which still needs to be turned from an implementation rule into a principled language-level specification.

19 Future Specification Work

A stronger next revision of this report should add:

- a dedicated annotation grammar,
- a normative section on storage layout and linkage,
- a source-level conversion table,
- a cleaned-up semantics of references versus pointers,
- a proper section on local functions and lambdas,
- a precise definition of `lis`,
- examples of legal and illegal programs.